

Documentation technique – SUPMEAL

GitHub : <https://github.com/Noha-SEVESTRE/4RESIT>

Ce document présente les informations nécessaires à l'installation, au lancement et au déploiement de l'application. Il décrit également l'architecture générale du projet et les technologies utilisées.

SUPMEAL est une application web permettant à un utilisateur de :

- Créer et gérer ses recettes
- Rechercher et filtrer ses recettes
- Ajouter des recettes à ses favoris
- Organiser ses repas dans un planning
- Créer des cookbook collaboratifs
- Gérer les membres et les permissions d'un cookbook
- Echanger des commentaires
- Importer et exporter ses données et des recettes
- Se connecter avec un compte local ou Google/GitHub

1. Présentation Générale

SUPMEAL repose sur une architecture composée de trois services principaux :

- Une application frontend en Vue.js
- Une API REST en Node.js, Express et TypeScript
- Une base de données PostgreSQL

La communication entre le frontend et le backend s'effectue principalement par requête HTTP.

Les messages et certaines modifications apportés aux cookbooks sont également transmis en temps réel grâce à Socket.IO

L'application est destinée à être utilisé depuis un navigateur web.

2. Prérequis

A. Déploiement avec Docker

Pour lancer l'application avec Docker, les outils suivants sont nécessaires :

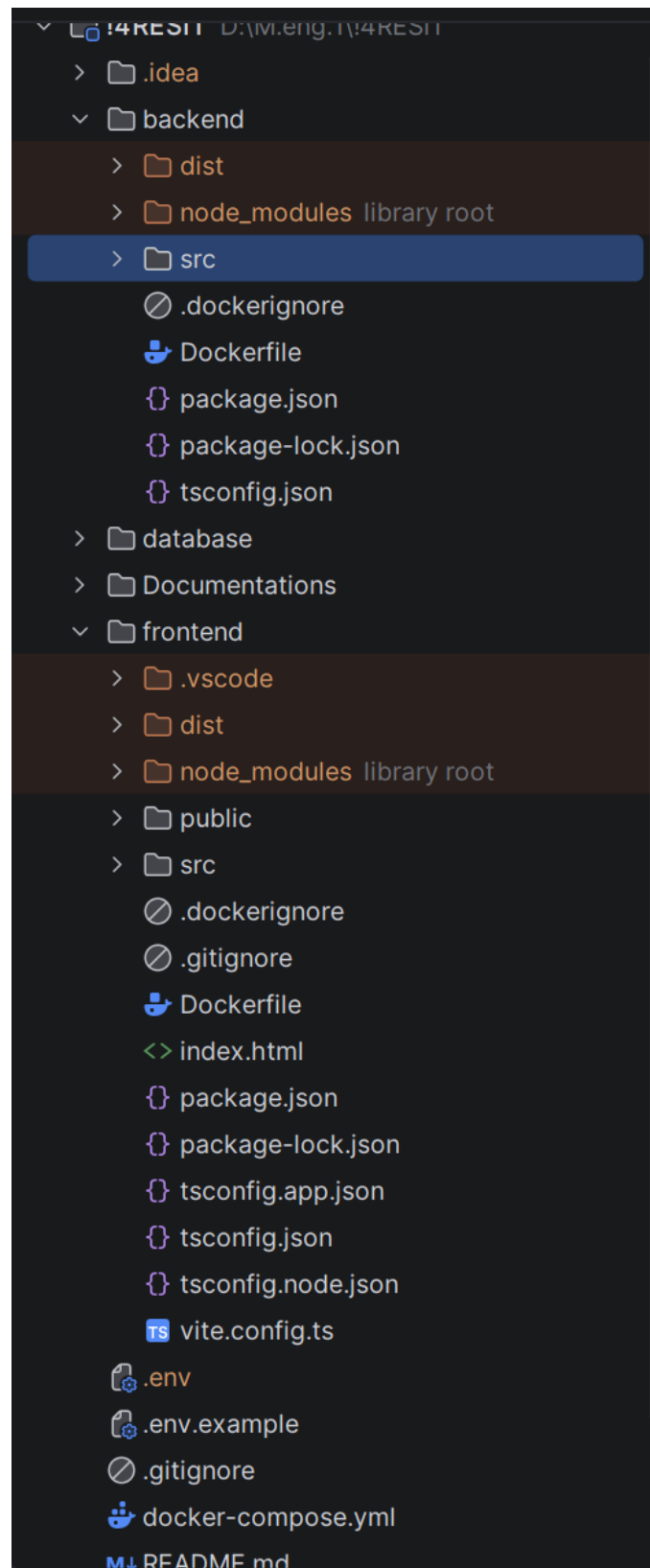
- Git
- Docker
- Docker compose
- Un navigateur web récent
- Un éditeur de code comme IntelliJ Idea ou VSC

Les versions utilisées par les conteneurs du projet sont :

- Node.js 22 Alpine pour le backend
- Node.js 22 Alpine pour le frontend
- PostgreSQL 16 Alpine pour la BDD

3. Structure générale du projet

Le projet est organisé de la manière suivante :



La répartition est la suivante :

- backend\ contient l'API REST, l'authentification, les règles métier et Socket.IO
- frontend\ contient l'interface utilisateur Vue.js
- database\ contient la création initiale des tables, relations et index
- docker-compose.yaml gère le frontend, le backend et PostgreSQL
- .env.example documente les variables d'environnement nécessaires

4. Variables d'environnement

Le projet utilise un fichier .env situé à la racine

Pour commencer, il faut copier le fichier d'exemple :

```
cp .env.example .env
```

ou pour Powershell Windows :

```
Copy-Item .env.example .env
```

A. Variables PostgreSQL

Variables	Exemple	Description
POSTGRES_DB	supmeal	Nom de la BDD
POSTGRES_USER	postgres	Utilisateur PostgreSQL
POSTGRES_PASSWORD	change_me	Mot de passe PostgreSQL
POSTGRES_HOST	Postgres	Nom du service PostgreSQL dans Docker
POSTGRES_PORT	5433	Port exposé sur la machine hôte

Dans Docker, PostgreSQL écoute sur le port interne 5432. La valeur correspond au port accessible depuis la machine hôte.

B. Variabel JWT

Variable	Exemple	Description
JWT_SECRET	Change_me_with_a_random_secret	Secret utilisé pour signer les tokens
JWT_EXPIRES_IN	7d	Durée de validité d'un Token

JWT_SECRET doit être remplacé par une valeur longue, aléatoire.

C. Ports et URLs

Variable	Exemple	Description
BACKEND_PORT	8080	Port exposé par l'API
FRONTEND_PORT	8081	Port exposé par l'application
FRONTEND_URL	http://localhost:8081	Origine autorisée pour Socket.IO et les redirections
VITE_API_URL	http://localhost:8080/api	Adresse de l'API utilisé par le frontend

D. Configuration GitHub OAuth

Variable	Description
GITHUB_CLIENT_ID	Identifiant de l'application OAuth Github
GITHUB_CLIENT_SECRET	Secret de l'application Github
GITHUB_CALLBACK_URL	URL de retour appelée après authentification

E. Configuration Google OAuth

Variable	Description
GOOGLE_CLIENT_ID	Identifiant OAuth Google
GOOGLE_CLIENT_SECRET	Secret OAuth Google
GOOGLE_CALLBACK_URL	URL de retour appelée après authentification

F. Exemple complet

POSTGRES_DB=supmeal

POSTGRES_USER=postgres

POSTGRES_PASSWORD=change_me

POSTGRES_HOST=postgres

POSTGRES_PORT=5433

JWT_SECRET=change_me_with_a_random_secret

JWT_EXPIRES_IN=7d

BACKEND_PORT=8080

FRONTEND_PORT=8081

FRONTEND_URL=http://localhost:8081

VITE_API_URL=http://localhost:8080/api

GITHUB_CLIENT_ID=change_me

GITHUB_CLIENT_SECRET=change_me

GITHUB_CALLBACK_URL=http://localhost:8080/api/auth/github/callback

GOOGLE_CLIENT_ID=change_me

GOOGLE_CLIENT_SECRET=change_me

GOOGLE_CALLBACK_URL=http://localhost:8080/api/auth/google/callback

5. Guide de déploiement avec Docker

A. Récupération du projet

git clone <https://github.com/Noha-SEVESTRE/4RESIT.git>

cd 4RESIT

Si vous utilisez le projet sous forme de .zip il vous suffit d'extraire le dossier puis de se placer à sa racine.

B. Configuration

Créer le .env :

cp .env.example .env

Modifier ensuite les valeurs sensibles :

- Mot de passe PostgreSQL
- Secret JWT
- Identifiants Google
- Identifiant Github
- URLs du frontend et du backend

C. Construction et démarrage

docker compose up --build

Cette commande :

- Télécharge l'image PostgreSQL
- Construit l'image du backend et du frontend
- Crée le réseau Docker
- Créer le volume PostgreSQL
- Initialise les tables
- Démarre les trois services

D. Vérification des conteneurs

`docker compose ps`

Les services suivants doivent être visibles :

- PostgreSQL
- Backend
- Frontend

E. Vérification de l'API

Vérification du serveur :

`curl http://localhost:8080/api/health`

Vérification de PostgreSQL :

`curl http://localhost:8080/api/db-health`

F. Accès aux interfaces

Service	Adresse
Application	http://localhost:8081
API REST	http://localhost:8080/api
Documentation swagger	http://localhost:8080/api-docs
PostgreSQL depuis l'hôte	localhost:5433

G. Consultation des journaux

Tous les services :

```
docker compose logs -f
```

Backend uniquement :

```
docker compose logs -f backend
```

Frontend uniquement :

```
docker compose logs -f frontend
```

PostgreSQL uniquement :

```
docker compose logs -f postgres
```

H. Arrêt de l'application

```
docker compose down
```

Cette commande arrête les conteneurs sans supprimer les données PostgreSQL.

I. Suppression complète des données

```
docker compose down -v
```

Cette commande supprime également le volume PostgreSQL. Toutes les données sont perdues.

J. Particularité du script init.sql

Ce script est exécuté lors de la première création du volume PostgreSQL.

S'il est modifié après l'initialisation, Docker ne l'exécute pas automatiquement une deuxième fois. Il faut alors :

- Appliquer manuellement les modifications
- Utiliser un système de migration
- Ou recréer le volume

6. Justification des choix technologiques

A. TypeScript

TypeScript est utilisé dans le backend et le frontend.

Il apporte :

- Une vérification statique des types
- Une meilleure autocomplétion
- Une détection plus rapide des erreurs
- Des contrats plus clairs entre l'API et l'interface
- Une maintenance plus simple

Il permet de conserver un langage commun sur l'ensemble de la partie applicative.

B. Node.js

Node.js permet d'exécuter JavaScript et TypeScript côté serveur.

Il est adapté à SUPMEAL grâce à :

- Sa bonne gestion des opérations réseau
- Son écosystème npm
- Sa comptabilité avec Socket.IO
- Sa simplicité pour développer une API REST
- Son intégration naturelle avec un frontend TypeScript

C. Express 5

Express est utilisé pour créer les routes HTTP de l'API.

Il a été choisi pour :

- Sa légèreté
- Sa prise en main rapide
- Sa flexibilité
- Sa comptabilité avec les middlewares
- Sa capacité à convenir à un projet comme SUPMEAL

D. Vue.js

Vue.js est utilisé pour construire l'interface web.

La composition API apporte :

- Des composants réactifs
- Une organisation claire de l'état
- Une syntaxe accessible
- Une intégration simple avec TypeScript
- Une bonne séparation entre affichage, données et événements

SUPMEAL utilise une navigation gérée principalement dans App.vue, sans Vue Router ni gestionnaire d'état externe. Ce choix limite le nombre de dépendances et reste adapté.

E. Vite

Vite est utilisé pour le développement et la compilation du frontend.

Ses avantages :

- Un démarrage rapide
- Le rechargement à chaud
- Une configuration légère
- Une bonne intégration avec Vue et TypeScript

F. PostgreSQL

PostgreSQL stocke les utilisateurs, recettes, cookbooks, membres, plannings, et discussions.

Il est adapté au projet grâce à :

- Son modèle relationnel
- La gestion des clés étrangères
- La prise en charge des transactions
- Ses contraintes d'intégrité
- Son support du type JSONB
- Ses capacités d'indexation et de recherche

G. Driver pg

Le projet utilise le pilote pg directement sans ORM.

Ce choix permet :

- De visualiser clairement les requêtes SQL
- De maîtriser les jointures
- De limiter les dépendances
- De rester proche du schéma de la base
- D'utiliser des requêtes paramétrées contre les injections SQL

L'inconvénient est que la logique SQL se trouve directement dans les modules de routes, ce qui rend la maintenance plus difficile si le projet grandit.

H. Zod

Zod valide les données reçus par l'API :

La bibliothèque vérifie notamment :

- Les adresses email
- Les longueur des textes
- Les UUID
- Les dates
- Les durées de préparation
- Les portions
- Les ingrédients
- Les étapes
- Les rôles des membres

Les données invalides sont rejetées avant l'exécution des requêtes SQL.

I. JSON Web Token

JWT est utilisé pour authentifier les requêtes.

Après la connexion, le backend produit un token signé contenant l'identifiant et l'adresse email de l'utilisateur. Le frontend envoie ensuite ce token dans l'en-tête :

Authorization: Bearer <token>

Cette méthode évite de gérer une session classique en mémoire sur le serveur.

J. BCryptJS

BcryptJS hache les mots de passe avec un facteur de coût de 10.

Aucun mot de passe local n'est enregistré en clair dans PostgreSQL.

K. Socket.IO

Socket.IO permet d'actualiser les échanges liés aux cookbooks sans rechargement manuel.

Il est utilisé pour diffuser :

- La création d'un message
- La suppression d'un message
- L'ajout d'un membre
- La modification d'un rôle
- Le retrait d'un membre
- L'ajout d'une recette
- Le retrait d'une recette

L. Swagger

Swagger documente les routes de l'API.

L'interface interactive est exposée sur :

/api-docs

Elle permet aux développeurs de consulter les endpoints et de tester les requêtes.

M. Docker

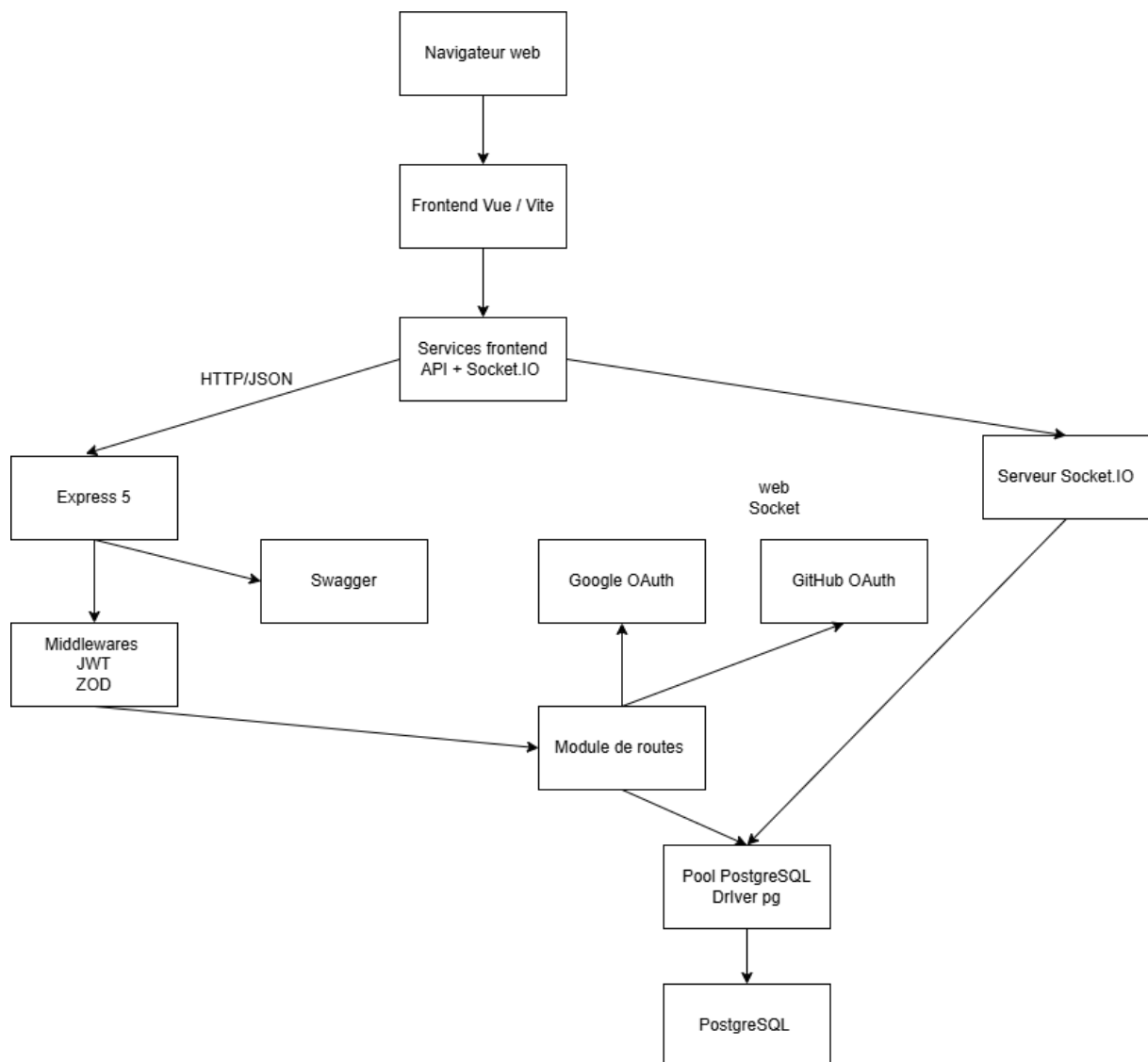
Docker garantit un environnement reproductible.

Il évite notamment :

- Les différences de version de PostgreSQL
- Les variations de configuration entre les machines
- L'installation manuelle de chaque service
- Les problèmes de ports

7. Architecture générale

A. Diagramme d'architecture



8. Architecture du Backend

A. Point d'entrée

Le fichier `server.ts` démarre le serveur http et initialise Socket.IO

Le fichier `app.ts` configure notamment :

- Express
- Helmet
- CORS
- La lecture JSON
- Swagger
- Les routes
- Le gestionnaire global des erreurs

B. Modules de routes

Le backend utilise une architecture volontairement simple.

Les fichiers de routes prennent en charge :

- La déclaration des endpoints
- La validation Zod
- Les règles métiers
- L'exécution des requêtes SQL
- La construction de la réponse http

C. Middlewares

Les middlewares principaux assurent :

- La lecture et la vérification du JWT
- L'injection de l'utilisateur authentifié dans la requête
- La sécurisation des en-têtes http
- La gestion des origines
- La journalisation des requêtes
- La validation des données

D. Accès aux données

La connexion est gérée avec un Pool PostgreSQL

```
import { Pool } from "pg";

export const pool = new Pool({
  host: process.env.POSTGRES_HOST,
  port: Number(process.env.POSTGRES_PORT ?? 5432),
  database: process.env.POSTGRES_DB,
  user: process.env.POSTGRES_USER,
  password: process.env.POSTGRES_PASSWORD
});
```

Les requêtes utilisent des paramètres :

SELECT *

FROM users

WHERE email = \$1;

Cette méthode évite de concaténer directement des valeurs utilisateur dans le SQL.

9. Architecture du frontend

A. Application Vue

Le frontend est une Single Page Application construite avec Vue.

App.vue joue un rôle central et gère :

- L'état d'authentification
- La section active
- Les données affichées
- L'ouverture de certains composants
- La navigation entre le tableau de board, les recettes, les cookbooks, le planning et les paramètres

B. Composants

Les composants principaux correspondent aux domaines fonctionnels :

- Authentification
- Tableau de board
- Recettes
- Formulaire de recette
- Détail d'une recette
- Planning
- Cookbooks
- Commentaires
- Messages
- Paramètres du compte

C. Service frontend

Les appels réseau sont regroupés dans les services spécialisés :

- Authentification
- Compte utilisateur
- Recettes
- Favoris
- Planning
- Cookbooks
- Discussions
- Import et Export

Les services utilisent l'API FETCH et ajoutent le token JWT aux routes protégées.

D. Stockage de la session

Le token est enregistré dans le navigateur avec la clé :

supmeal_token

Il est actuellement stocké dans localStorage

E. Images des recettes

Le frontend autorise l'ajout d'une image locale.

Le fichier est converti en URL base 64 et enregistré dans le champ `image_url` de la recette.

La limite appliquée par l'interface est de 1.5Mo.

10. Authentification et sécurité

A. Création d'un compte local

L'inscription demande :

- Une adresse email
- Un mot de passe
- Un nom d'affichage

Le mot de passe est haché avec BCryptJS avant son insertion en base.

B. Connexion locale

Le backend :

- Recherche l'utilisateur par email
- Vérifie que le compte possède un mot de passe
- Contrôle l'éventuel verrouillage
- Compare le mot de passe avec le hash
- Génère un token JWT en cas de réussite

C. Protection contre les tentatives répétées

Après 5 échecs successifs, le compte est verrouillé pendant cinq minutes.

Les champs utilisés sont :

- `Failed_login_attempts`
- `Locked_until`

D. OAuth

Les connexions OAuth sont implémentées directement à l'aide des endpoints des fournisseurs.

Lors du retour OAuth :

- Le fournisseur transmet l'identité de l'utilisateur
- SUPMEAL récupère son email
- Un compte OAuth existant est recherché
- Sinon le compte est relié à un utilisateur possédant le même email
- A défaut, un nouvel utilisateur est créé
- Un token JWT est généré

Les utilisateurs OAuth ont un password_hash nul.

Ils ne peuvent donc pas modifier leur mot de passe ou leur email depuis SUPMEAL.

E. Protections actuellement présentes

- Hachage BcryptJS
- JWT signé
- Validation Zod
- Requêtes SQL paramétrées
- Helmet
- Contrôle des permissions par cookbook
- Durée de verrouillage après échec
- Contrôle d'appartenance avant de rejoindre une room Socket.IO

11. Gestion des rôles dans les cookbooks

Chaque membre possède un rôle propre.

Fonctionnalité	Propriétaire	Editeur	Commentateur	Lecteur
Consulter le cookbook	Oui	Oui	Oui	Oui
Consulter les recettes	Oui	Oui	Oui	Oui
Ajouter une recette	Oui	Oui	Non	Non
Retirer une recette	Oui	Oui	Non	Non
Publier un commentaire	Oui	Oui	Oui	Non
Publier un message	Oui	Oui	Oui	Non
Supprimer son propre contenu	Oui	Oui	Oui	Non
Supprimer le contenu des autres	Oui	Non	Non	Non
Ajouter un membre	Oui	Non	Non	Non
Modifier le rôle d'un membre	Oui	Non	Non	Non
Retirer un membre	Oui	Non	Non	Non
Supprimer le cookbook	Oui	Non	Non	Non

Le rôle PROPRIETAIRE est automatiquement attribué à la personne qui crée le cookbook.

Les rôles ne sont pas globaux : un utilisateur peut être PROPRIETAIRE dans un cookbook et LECTEUR dans un autre.

12. Communication en temps réel

A. Authentification Socket.IO

Le frontend transmet son token lors de l'ouverture de la connexion Socket.IO

Le backend vérifie le JWT avant d'accepter la connexion.

B. Rooms de cookbooks

Chaque cookbook possède une room logique :

cookbook:<uuid>

Avant qu'un client rejoigne la room, le serveur vérifie son appartenance dans `cookbook_members`.

C. Événements principaux

Les événements actuellement diffusés comprennent notamment :

cookbook:message-created

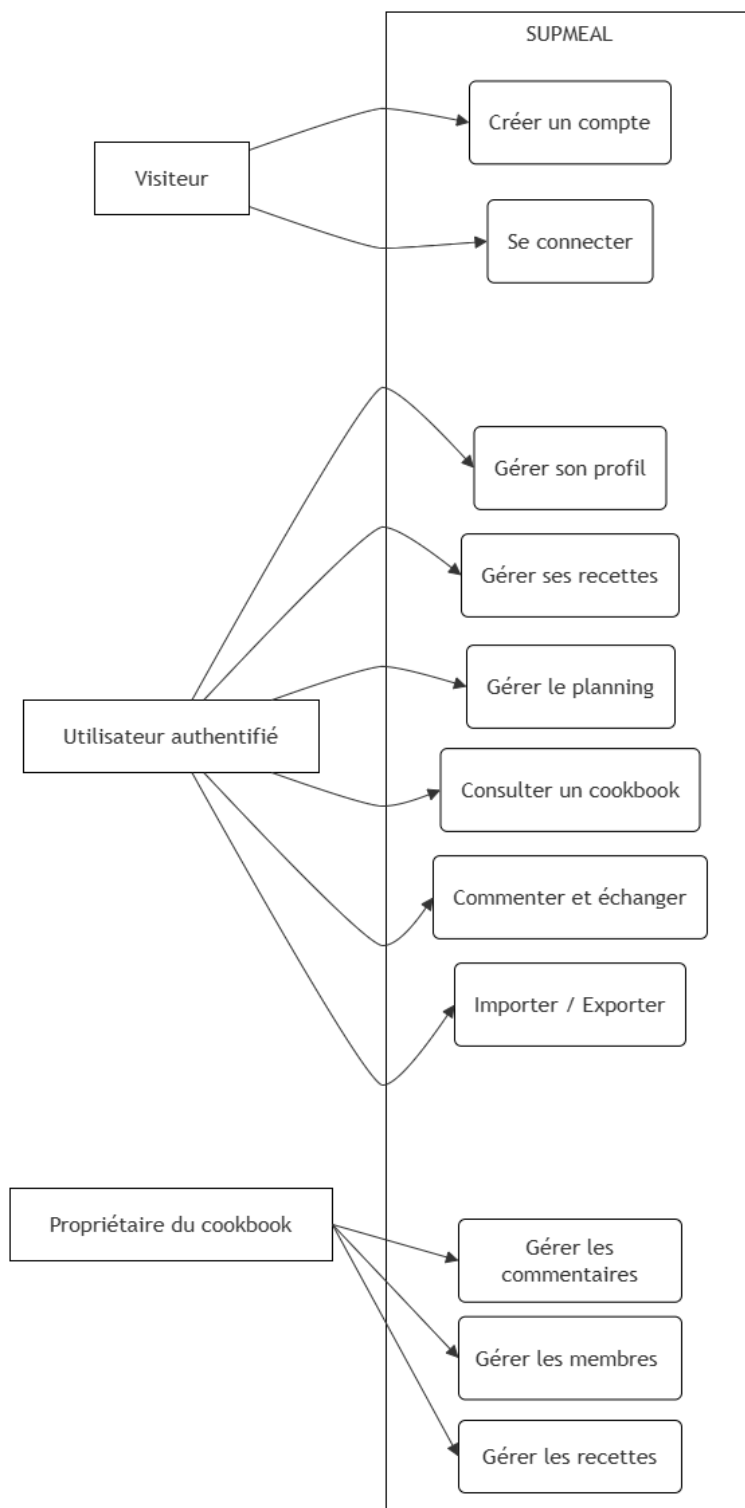
cookbook:message-deleted

cookbook:updated

L'événement `cookbook:updated` peut correspondre à :

- Un membre ajouté ou modifié
- Un membre retiré
- Une recette ajoutée
- Une recette retirée

13. Diagramme UML des cas d'utilisation



14. Routes de l'API

A. Routes publiques et techniques

Méthodes	Routes	Description
GET	/api/health	Vérifie que l'API répond
GET	/api/db-health	Vérifie la connexion PostgreSQL
GET	/api-docs	Affiche Swagger

B. Authentification

Méthode	Route	Protection	Description
POST	/api/auth/register	Non	Créer un compte
POST	/api/auth/login	Non	Connecte un utilisateur
GET	/api/auth/me	JWT	Retourne l'utilisateur connecté
GET	/api/auth/google	Non	Démarre OAuth Google
GET	/api/auth/github	Non	Démarre Github
GET	/api/auth/google/callback	Non	Traite le retour Google
GET	/api/auth/github/callback	Non	Traite le retour Github

C. Compte utilisateur

Méthode	Route	Description
GET	/api/users/me/preferences	Récupère le profil et les préférences
PATCH	/api/users/me/preferences	Met à jour le profil et les préférences
GET	/api/users/me/security	Retourne les informations de sécurité du compte
PATCH	/api/users/me/email	Modifie l'adresse email d'un compte
PATCH	/api/users/me/password	Modifie le mot de passe d'un compte

D. Recettes

Méthode	Route	Description
GET	/api/recipes	Liste les recettes avec filtres
POST	/api/recipes	Crée une recette
GET	/api/recipes/favorites	Liste les recettes favorites
GET	/api/recipes/ :id	Retourne une recette
PUT	/api/recipes/ :id	Modifie une recette
DELETE	/api/recipes/ :id	Supprime une recette
POST	/api/recipes/ :id/favorite	Ajoute une recette aux favoris
DELETE	/api/recipes/ :id/favorite	Retire une recette des favoris
GET	/api/recipes/ :id/export	Exporte une recette
POST	/api/recipes/import	Importe une recette

E. Planning

Méthode	Route	Description
GET	/api/meal-plans	Liste les repas planifiés
POST	/api/meal-plans	Ajoute une recette au planning
DELETE	/api/meal-plans/ :id	Retire une entrée du planning

F. Cookbooks

Méthode	Route	Description
GET	/api/cookbooks	Liste des cookbooks accessibles
POST	/api/cookbooks	Créer un cookbook
GET	/api/cookbooks/ :id	Retour le détail d'un cookbook
DELETE	/api/cookbooks/ :id	Supprime un cookbook
POST	/api/cookbooks/ :id/members	Ajoute un membre ou modifie son rôle
DELETE	/api/cookbooks/ :id/members/:userID	Retire un membre
GET	/api/cookbooks/:id/recipes	Liste des recettes du cookbook
POST	/api/cookbooks/ :id/recipes/ :recipes	Ajoute une recette
DELETE	/api/cookbooks/:id/recipes/:recipeId	Retire une recette

G. Discussions

Méthode	Route	Description
GET	/api/recipes/:recipeId/comments	Liste les commentaires d'une recette
POST	/api/recipes/:recipeId/comments	Ajoute un commentaire
DELETE	/api/recipes/:recipeId/comments/:commentId	Supprime un commentaire
GET	/api/cookbooks/:cookbookId/messages	Liste les messages d'un cookbook
POST	/api/cookbooks/:cookbookId/messages	Envoie un message
DELETE	/api/cookbooks/:cookbookId/messages/:messageId	Supprime un message

H. Import et Export

Méthode	Route	Description
GET	/api/data/export	Exporte
POST	/api/data/import	Importe

15. Gestion des recettes

Une recette contient :

- Un titre
- Une description
- Une durée de préparation
- Une durée de cuisson
- Un nombre de portion
- Une image facultative
- Une source facultative
- Au moins un ingrédient
- Au moins une étape
- Des tags facultatifs

A. Contraintes principales

Champ	Contraintes
Titre	Obligatoire, maximum 180 caractères
Description	Facultative, longueur limitée
Préparation	De 0 à 1440 minutes
Cuisson	De 0 à 1440 minutes
Portions	De 1 à 50
Ingrédients	Au moins 1
Etapes	Au moins 1
Tags	Maximum 20
Image	Maximun 1.5Mo

B. Recherches

La route de recherche peut filtrer selon :

- Une chaine libre
- Un tag
- Un ingrédient
- Le temps total maximal
- Le temps de préparation maximal
- Le temps de cuisson maximal
- Le nombre minimum de portion
- Le statut favori
- Le cookbook
- Les recettes personnelles

La chaine libre recherche dans plusieurs éléments :

- Titre
- Description
- Tags
- Ingrédients
- Etapes

C. Suppression

La suppression d'une recette est directe. SUPMEAL ne possède pas de corbeille.

Les ingrédients, étapes, tags, favoris et commentaires liés sont supprimés par cascade.

16. Planning des repas

Le planning relie :

- Un utilisateur
- Une recette
- Une date
- Un type de repas

Les types proposés par l'interface sont :

- Petit déjeuner
- Déjeuner
- Diner
- Collation

La suppression d'une entrée du planning ne supprime pas la recette.

17. Import et Export

A. Export d'une recette

L'export individuel produit un fichier JSON identifié par

SUPMEAL_RECIPES_EXPORT.

Le fichier contient les informations nécessaires à la reconstruction de la recette.

B. Import d'une recette

Le backend vérifie :

- Le type d'export
- La version
- Le titre
- Les durées
- Les portions
- Les ingrédients
- Les étapes
- Les tags

L'import crée une nouvelle recette. Il ne remplace pas automatiquement une recette existante.

C. Export global

L'export global utilise le type

SUPMEAL_FULL_EXPORT

Il contient principalement :

- Les recettes personnelles
- Les cookbooks accessibles
- Les recettes présentes dans les cookbooks

D. Limite de l'import global

L'import global :

- Ajoute les données au compte
- Ne supprime pas les données existantes
- Ne déduplique pas automatiquement les recettes
- Crée les cookbooks importés au nom de l'importateur
- N'importe pas les mots de passes
- N'importe pas les membres
- N'importe pas les commentaires
- N'importe pas les messages
- N'importe pas les favoris
- N'importe pas le planning

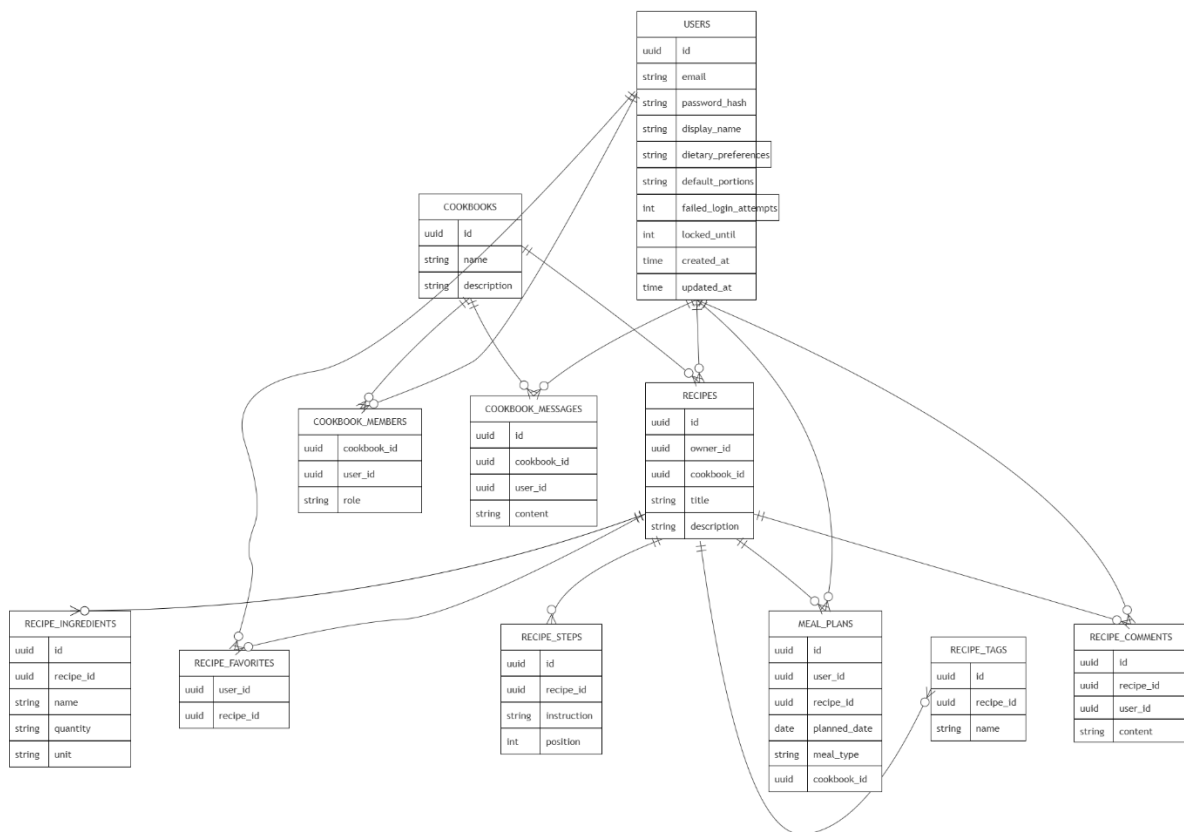
Les fichiers d'export sont des fichiers JSON lisibles.

18. Base de données

A. Tables principales

Table	Rôle
Users	Compte utilisateur
Oauth_accounts	Liaison Google et Github
Cookbooks	Espace collaboratifs
Cookbook_membre	Membres et rôles
Recipes	Informations générales des recettes
Recipes_ingredients	Ingrédients
Recipes_steps	Etapes
Recipes_tags	Tags
Recipe_favorite	Favoris des utilisateurs
Meal_plans	Repas planifiés
Recipes_comments	Commentaires sur les recettes
Cookbook_messages	Messages sur les cookbooks

B. Schéma relationnel



C. Contraintes importantes

- User.email est unique
- Un fournisseur et son identifiant distant forment une combinaison unique
- Un membre ne peut apparaître qu'une fois dans un cookbook
- Le rôle doit appartenir à la liste Propriétaire, éditeur, lecteur, commentateur
- Une recette possède un propriétaire
- Une recette peut être associée à un cookbook
- Un favori est unique pour un couple utilisateur-recette
- Les suppressions utilisent principalement les cascades PostgreSQL

D. Index

Le script SQL ajoute notamment des index sur :

- Le propriétaire d'une recette
- Le titre d'une recette
- Le nom des ingrédients
- Le nom des tags
- L'utilisateur et la date d'un planning
- La recette des commentaires
- Le cookbook des messages

Ces index améliorent les recherches et les jointures les plus fréquentes.